Ingeniería del Software II

Patrones de Diseño

- Patrones de Diseño. Aplicación de Graps. Conceptos
- Fundamentales. Características principales de los distintos Patrones. Diseño de la realización del caso de uso con patrones de diseño. Adaptador. Factoría.
- Singleton. Estrategia. Composite. Fachada. Observador/Publicar/Suscribir/
- Modelos de Delegación de Eventos. Ventajas sobre su aplicación.

Introducción.

- El diseño es un modelo del sistema, realizado con una serie de principios y técnicas, que permite describir el sistema con el suficiente detalle como para ser implementado. Pero los principios y reglas no son suficientes, en el contexto de diseño podemos observar que los buenos ingenieros tienen esquemas y estructuras de solución que usan numerosas veces en función del contexto del problema. Estos esquemas y estructuras son conceptos reusables y nos permiten reutilizar un esquema de solución ante problemas similares.

Introducción.

- "Una arquitectura orientada a objetos bien estructurada está llena de patrones. La calidad de un sistema orientado a objetos se mide por la atención que los diseñadores han prestado a las colaboraciones entre sus objetos. Los patrones conducen a arquitecturas más pequeñas, más simples y más comprensibles". (Grady Booch)

Hacer software no es fácil

- Diseñar software orientado a objetos es difícil, y diseñar software orientado a objetos reutilizable es todavía más difícil.

  - Chapter 1: Introduction. Design Patterns, *The Gang of Four*

…y un software capaz de evolucionar tiene que ser reutilizable (al menos para las versiones futuras)

Diseñar para el cambio

- El software cambia
- Para anticiparse a los cambios en los requisitos hay que diseñar pensando en qué aspectos pueden cambiar
- Los patrones de diseño están orientados al cambio

Patrones

Cómo llegar a ser un maestro de ajedrez

- Primero aprender las reglas del juego
- nombres de las piezas, movimientos legales, geometría y orientación del tablero, etc.
- A continuación aprender los principios
- relativo valor de las piezas, valor estratégico de las casillas centrales, jaque cruzado, etc.
- Sin embargo, para llegar a ser un maestro, hay que estudiar las partidas de otros maestros
- Estas partidas contienen patrones que deben ser entendidos, memorizados y aplicados repetidamente
- Hay cientos de estos patrones

Patrones

- Christopher Alexander (1977):

Cada patrón describe un problema que ocurre una y otra vez en nuestro entorno, y describe la esencia de la solución a ese problema, de tal modo que pueda utilizarse esta solución un millón de veces más, sin siquiera hacerlo de la misma manera dos veces

- Un patrón es:
  - una solución a un problema en un contexto particular
  - recurrente (lo que hace la solución relevante a otras situaciones)
  - enseña (permite entender cómo adaptarlo a la variante particular del problema donde se quiere aplicar)
  - tiene un nombre para referirse al patrón

- Los patrones facilitan la reutilización de diseños y arquitecturas software que han tenido éxito

Motivación de los Patrones

- Capturan la experiencia y la hacen accesible a los no expertos
- El conjunto de sus nombres forma un vocabulario que ayuda a que los desarrolladores se comuniquen mejor
- Lenguajes de patrones
- Ayudan a la gente a comprender un sistema más rápidamente cuando está documentado con los patrones que usa
- Los patrones pueden ser la base de un manual de ingeniería de software
  - *Si el software se convierte en una ingeniería, las prácticas exitosas deben ser documentadas sistemáticamente y ampliamente difundidas*

Motivación de los Patrones

- Facilitan la reestructuración de un sistema tanto si fue o no concebido con patrones en mente
- Reutilización:
- Los patrones de diseño soportan la reutilización de arquitecturas software
- Los armazones soportan la reutilización del diseño y del código
- El software cambia
- Para anticiparse a los cambios en los requisitos hay que diseñar pensando en qué aspectos pueden cambiar
- Los patrones de diseño están orientados al cambio

Clasificación

- Patrones *arquitecturales*

Expresan un paradigma fundamental para estructurar un sistema software

Proporcionan un conjunto de subsistemas predefinidos, con reglas y guías para organizar las relaciones entre ellos.

- Patrones *de diseño*

Compuestos de varias unidades arquitecturales más pequeñas.

Describen el esquema básico para estructurar subsistemas y componentes

- Patrones elementales (*idioms)*

Específicos de un lenguaje de programación

Describen cómo implementar componentes particulares de un patrón

Ejemplos de patrones

Patrones arquitecturales

- Jerarquía de capas
- Tuberías y filtros
- Cliente/Servidor
- Maestro-Esclavo
- Control centralizado y distribuido

Patrones de diseño

- Proxies
- Factorías
- Adaptadores
- Composición
- Broker

Patrones elementales

- Modularidad
- Interfaces mínimas
- Encapsulación
- Objetos
- Acciones y Eventos
- Concurrencia

Patrones y Armazones

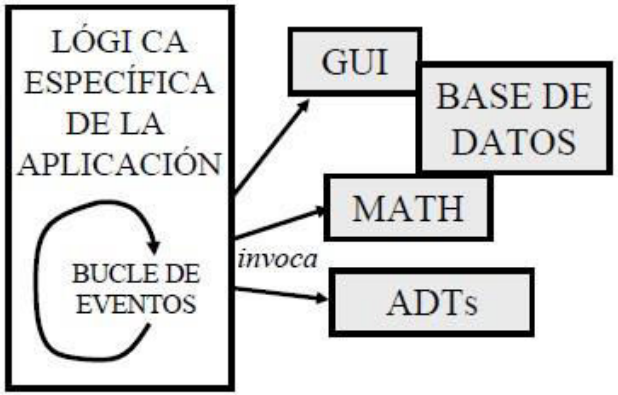
- Los patrones de diseño tienen descripciones más abstractas que los armazones
- Las descripciones de patrones suelen ser independientes de los detalles de implementación o del lenguaje de programación (salvo ejemplos usados en su descripción)
- Los armazones están implementados en un lenguaje de programación, y pueden ser ejecutados y reutilizados directamente
- Los patrones de diseño son elementos arquitecturales más pequeños que los armazones
- Un armazón incorpora varios patrones
- Los patrones se pueden usar para documentar armazones
- Los patrones de diseño están menos especializados que los armazones
- Los armazones siempre se aplican a un dominio de aplicación particular

Armazones (Frameworks)

- Características
- Un armazón ofrece un conjunto integrado de funcionalidad específica de un dominio
- P.ej.: aplicaciones financieras, servicios de telecomunicación, sistemas de ventanas, bases de datos, aplicaciones distribuidas, núcleos de SO
- Los armazones invierten el control en ejecución entre la aplicación y el software sobre el que está basada
- El armazón determina qué métodos se invocan en respuesta a eventos (se reusa el código del cuerpo principal y se escribe el código al que llama)
- Un armazón es una aplicación medio-acabada
- Las aplicaciones completas se desarrollan mediante herencia, e instanciando componentes parametrizados del armazón

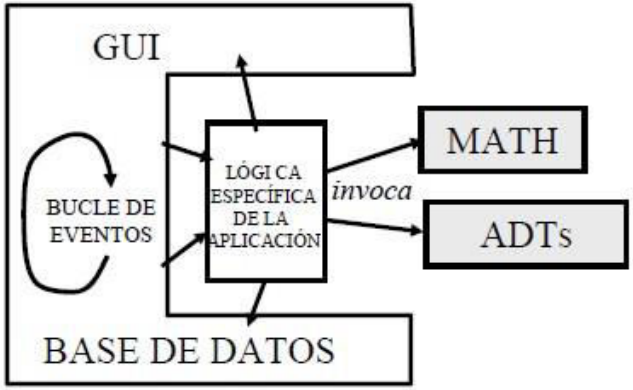
Armazones (*Frameworks*)

Diferencia de Bibliotecas



Arquitectura basada en biblioteca de clases

☞ ***reutilización de código***



Arquitectura basada en armazón

☞ ***reutilización de diseño y código***

Clasificación

- Los patrones de diseño se clasifican según el propósito para el que han sido definidos:
- **Creacionales:** solucionan problemas de creación de instancias. Nos ayudan a encapsular y abstraer dicha creación. Tratan de la inicialización y configuración de clases y objetos.
- **Estructurales:** solucionan problemas de composición (agregación) de clases y objetos. Tratan de desacoplar interfaz e implementación de clases y objetos: ¿Cómo se componen clases y objetos?
- **De Comportamiento**: soluciones respecto a la interacción y responsabilidades entre clases y objetos, así como los algoritmos que encapsulan. Tratan de las interacciones dinámicas entre sociedades de clases y objetos: ¿Cómo interaccionan y se distribuyen responsabilidades los objetos?

Clasificación

- Según el ámbito se clasifican en patrones de clase y de objeto:

		Propósito		
		Creación	*Estructural*	*Comportamiento*
Ámbito	*Clase*	Factory Method	Adapter	Interpreter Template Method
	Objeto	Abstract Factory Builder Prototype Singleton	Adapter Bridge Composite Decorator Facade Flyweight Proxy	Chain of Responsability Command Iterator Mediator Memento Observer State Strategy Visitor

Características.

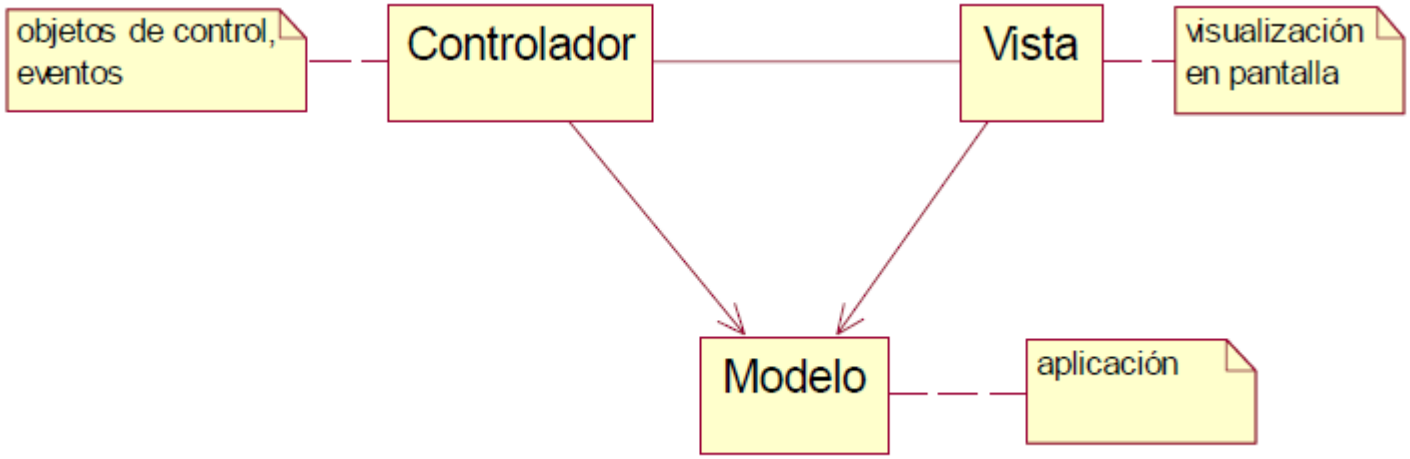
- Los patrones de diseño son **descripciones de clases** cuyas **instancias colaboran entre sí**. Cada patrón es adecuado para ser adaptado a un cierto tipo de problema. Para describir un caso debemos especificar[GAM95]::
  - Nombre; Por ejemplo Filtro.
  - Propósito o finalidad, Por ejemplo, un patrón debe tener la intención de facilitar el mantenimiento
  - Sinónimos (otros nombres por los que puede ser conocido)

Características.

- Problema al que es aplicable
- Estructura (diagrama de clases)
- Participantes (responsabilidad de cada clase)
- Colaboraciones (diagrama de interacciones)
- Implementación (consejos, notas y ejemplos)
- Otros patrones con los que está relacionado

Ejemplo de patrón de diseño

- Modelo-Vista-Controlador
  - Utilizado en Smalltalk [Krasner and Pope, 1988]
  - Distribución de responsabilidades
  - Utilizado en JFC (componentes Swing).



Ventajas.

- La clave para la reutilización es anticiparse a los nuevos requisitos y cambios, de modo que los sistemas evolucionen de forma adecuada.

- Cada patrón permite que algunos aspectos de la estructura del sistema puedan cambiar independientemente de otros aspectos.

Ventajas.

- Un patrón es un **esquema o microarquitectura** que supone una solución a problemas (dominios de aplicación) semejantes.

- Los patrones facilitan la reusabilidad, extensibilidad y mantenimiento.

Descripción de patrones de diseño

- Nombre e intención del patrón

Referencia al patrón

Incrementa el vocabulario de diseño

Problema y contexto

Cuándo aplicar el patrón

Solución

Estructura: elementos que conforman el diseño, sus relaciones, responsabilidades y colaboraciones es una descripción abstracta de cómo una disposición de elementos (clases y objetos) solucionan el problema

Se ilustra con un ejemplo de código

Consecuencias (positivas y negativas)

Necesidades (tiempo, memoria), aspectos de implementación y lenguaje de programación, flexibilidad, extensibilidad, portabilidad

Patrones relacionados

Cómo seleccionar un patrón de diseño

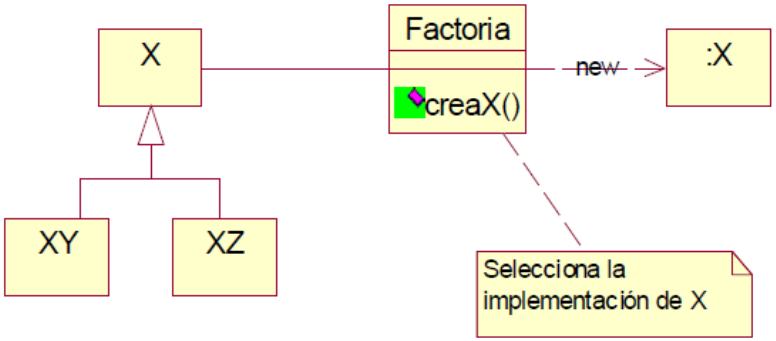
- Considerar cómo los patrones de diseño solucionan problemas de diseño
- Buscar las intenciones de cada patrón
- Estudiar cómo se interrelacionan los patrones
- Estudiar patrones de propósito similar
- Examinar la causa de un rediseño
- Considerar qué debería ser variable en un diseño

Cómo usar un patrón de diseño

1. Leer el patrón una vez para tener una visión general
2. Volver y estudiar la estructura, los participantes y las colaboraciones
3. Ver un ejemplo concreto codificado del patrón
4. Elegir nombres para los participantes del patrón que sean significativos en el contexto de la aplicación
5. Definir las clases
6. Definir nombres específicos de la aplicación para las operaciones en el patrón
7. Implementar las operaciones que realizarán las responsabilidades y colaboraciones del patrón

Patrones de creación

- En Java para crear un objeto se utiliza el operador new
  unObjeto = new ClaseObjetos ();
- Una clase ***Factoría*** *puede ocuparse de los detalles de qué* clase de implementación utilizar para crear un objeto de un tipo determinado



Patrones de creación

- Permiten que el sistema sea independiente de cómo se crean, componen o representan sus objetos.
- Sistemas más dependientes de la composición de objetos que de la herencia de clases
- Se trata de que el comportamiento se defina más por la composición de un conjunto pequeño de comportamientos fundamentales que por la definición mediante herencia de todos los comportamientos posibles
- Por tanto, la creación de objetos es algo más que instanciar una clase

Patrones de creación

- Temas recurrentes en los patrones de creación:
  - Encapsulan el conocimiento sobre las clases concretas que se van a utilizar
  - Ocultan la manera de crear objetos de estas clases y cómo se juntan
- La visión global de los objetos del sistema son sus interfaces (que pueden definirse como clases abstractas o interfaces)
- Se da independencia de
  - qué se crea
  - quién lo crea
  - cómo se crea
  - cuándo se crea

Tipos de patrones de creación

- De clase: usa herencia para variar la clase del objeto creado.
  - Factoría abstracta
  - Factoría para construir familias de objetos
  - Builder
  - Factoría para construir objetos complejos de forma incremental
- De objeto: delega la creación en otro objeto

Método Factoría

- Interfaz que permite que sean las subclases las que determinen qué clase instanciar

Prototype

- Factoría para clonar nuevos ejemplares copiando de un prototipo

Singleton

Object Pool

- Factoría que asegura que sólo hay un miembro (singleton) o un conjunto determinado (object pool) de una clase, y proporciona un punto global de acceso a él.

Método Factoría

- Propósito:

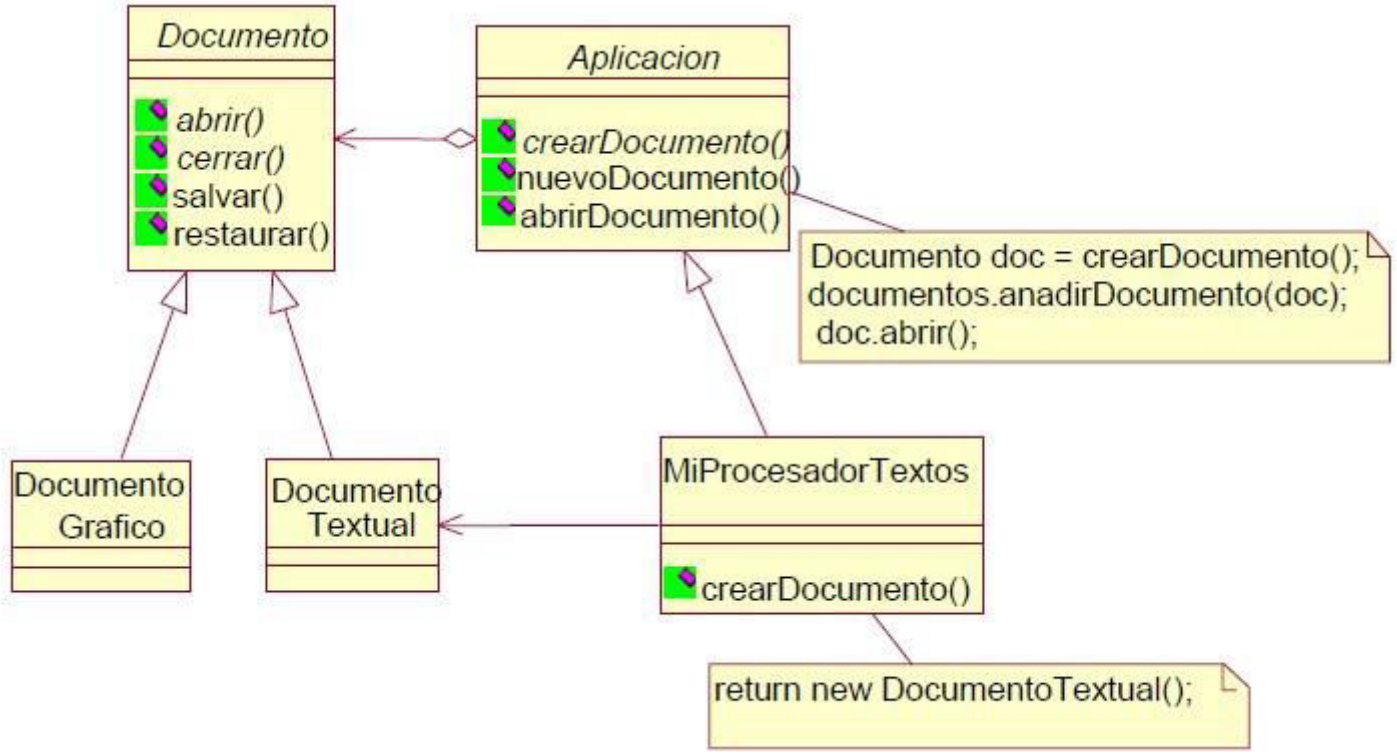
Permite que una clase difiera la instanciación a las subclases (son éstas las que deciden qué clase instanciar)

- Otras denominaciones:
  - *Factory Method*
  - *Virtual Constructor*

Motivación

- Un armazón utiliza clases abstractas para definir y mantener relaciones entre objetos
- El armazón también es responsable de crear esos objetos
- En principio el armazón no conoce las clases concretas que se van a definir en una aplicación concreta
- El método factoría es un método abstracto que encapsula el conocimiento de qué subclase se va a crear y pone este conocimiento fuera del armazón

Método Factoría

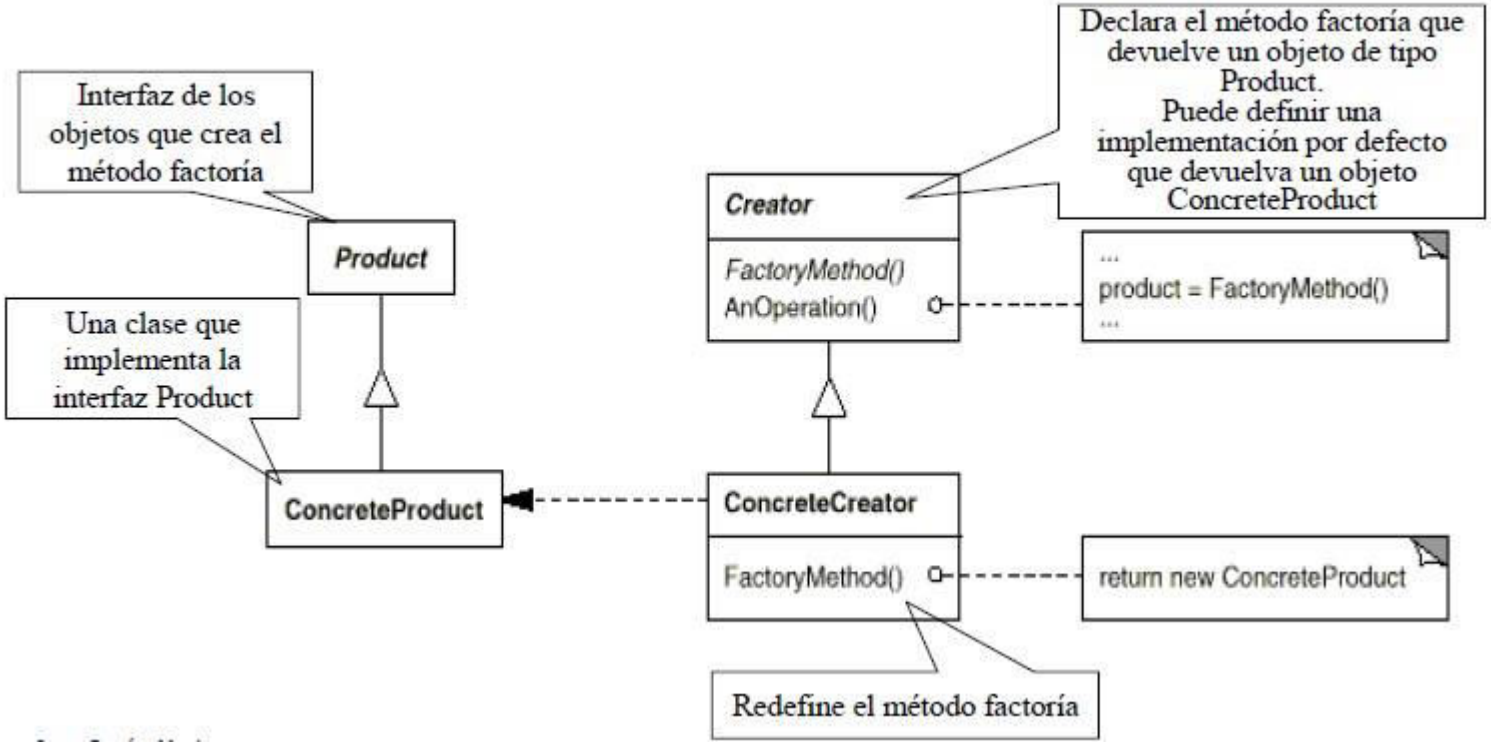


Método Factoría

- Aplicación
  - Cuando una clase no puede anticipar el tipo de objetos que debe crear
  - Cuando una clase quiere que sus clases especifiquen los objetos que deben crear
  - Cuando una clase delega la responsabilidad a una subclase de ayuda (entre varias), y se quiere localizar el conocimiento de qué subclase de ayuda es la delegada

Método Factoría

- Esquema, participantes y colaboraciones
  - El Creator deja que sus subclases definan el método factoría que construye y devuelve un objeto del ConcreteProduct apropiado



Método Factoría

Consecuencias

- Elimina la necesidad de enlazar clases específicas de la aplicación en el código
- Es más flexible crear un objeto con un método factoría que directamente

- Un método factoría puede dar una implementación por defecto, por ejemplo, para crear un archivo, mostrar un diálogo que permita abrir uno ya existente
  - En este ejemplo el método factoría no es abstracto
  - Conectar jerarquías de clases paralelas
- Los métodos factoría pueden ser llamados por los clientes, no sólo por los Creator
  - *Ejemplo: manipuladores de figuras: un tipo de manipulador* con varios métodos factoría para cada tipo de figura

Método Factoría

- Implementación
  - Implementación de la clase Creator
  - El método factoría es abstracto
  - El método factoría proporciona una implementación por defecto:
  - Permite extensibilidad: se pone la creación de objetos en una operación separada por si el usuario quiere cambiarla
- Métodos factoría parametrizados
  - Para crear distintos tipos de objetos

```
public class Creator {
public Product create(ProductId);
};
```

Factoría Abstracta

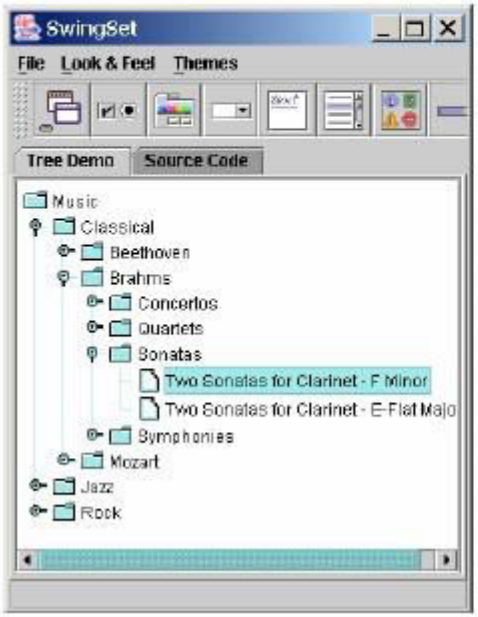
Propósito:

- Proporcionar una interfaz para crear familias de objetos relaciones o dependientes, sin especificar sus clases concretas
- Otras denominaciones:
  - Abstract Factory
  - Kit
  - Motivación
- Definir una interfaz que soporte diferentes sistemas de ventanas, p.ej. Motif, Windows, Openview, …

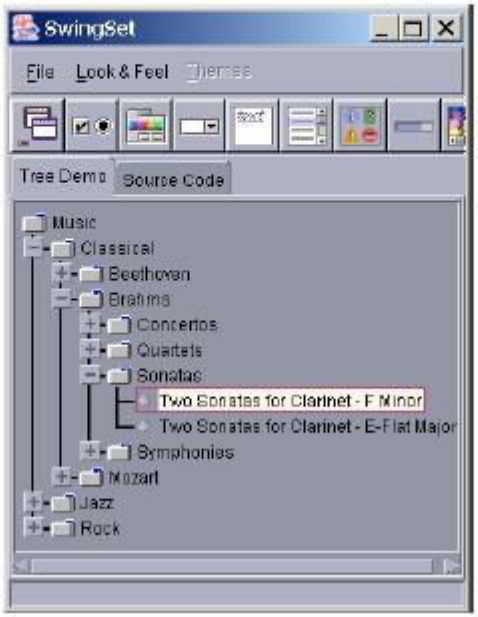
Factoría Abstracta

- Varias apariencias (Look & Feel) para la interfaz gráfica:

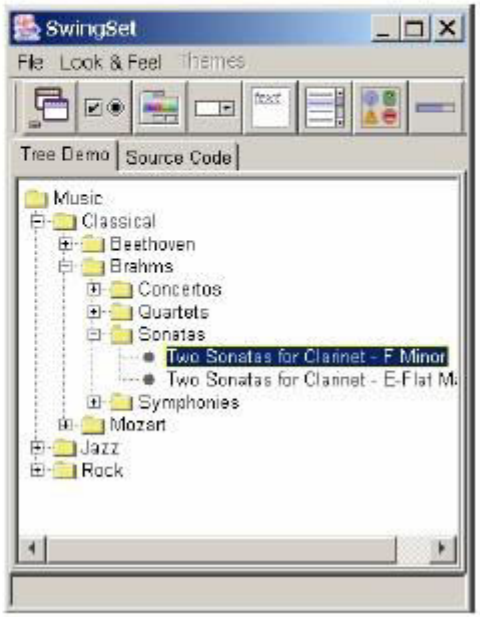
JAVA L&F



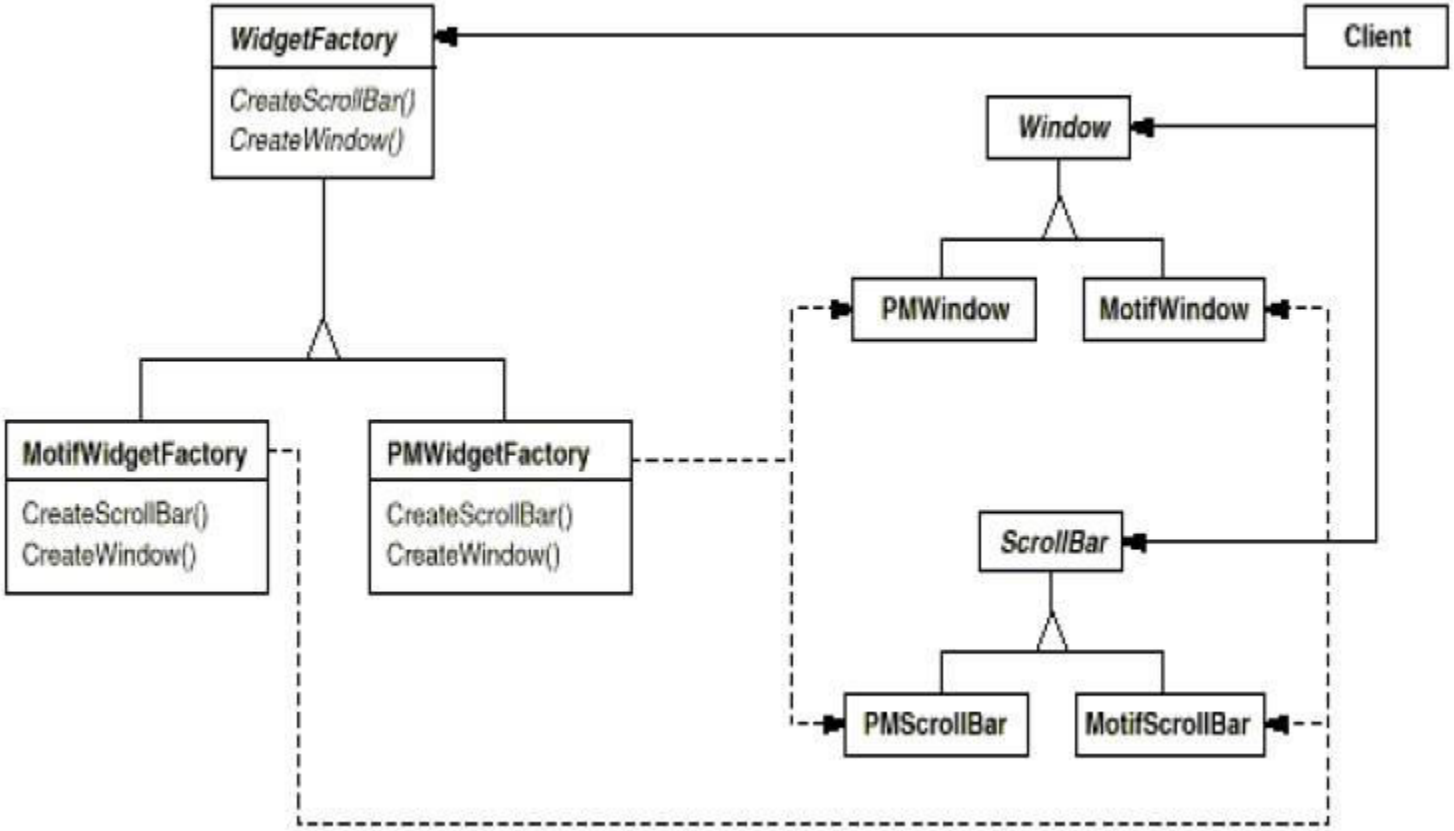
Motif L&F



Windows L&F



Factoría Abstracta



Factoría Abstracta

Aplicación

- Para sistemas independientes de cómo se crean, componen y representan sus productos
- Para sistemas que pueden configurarse con una de varias familias de productos
- Para obligar a utilizar juntos objetos de una familia de productos
- Para ofrecer una librería de clases, mostrando sólo sus interfaces y no sus implementaciones

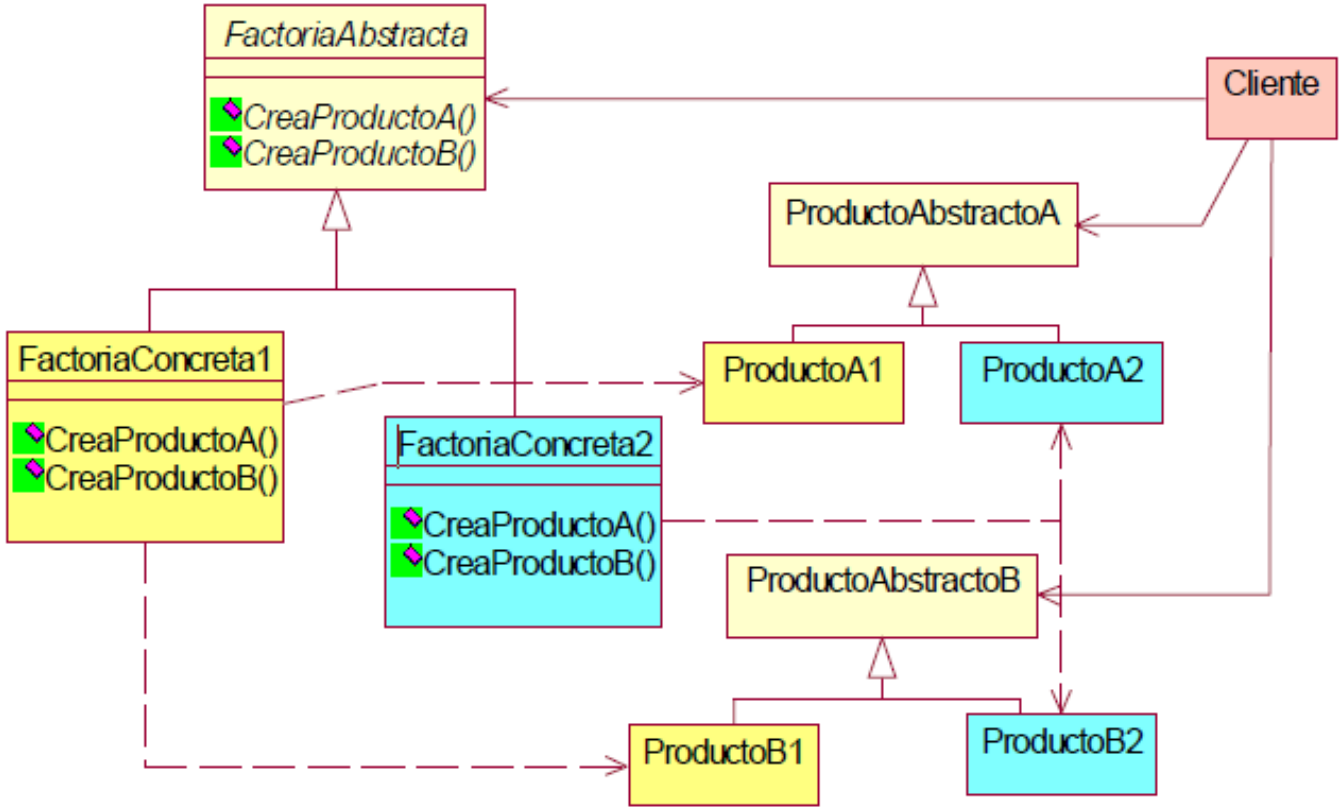
Factoría Abstracta

Consecuencias

- Se oculta a los clientes las clases de implementación
  - los clientes manipulan los objetos a través de las interfaces o clases abstractas
- Facilita el intercambio de familias de productos
  - Al crear una familia completa de objetos con una factoría abstracta, es fácil cambiar toda la familia de una vez simplemente cambiando la factoría concreta
- Mejora la consistencia entre productos
  - El uso de la factoría abstracta permite forzar a utilizar un conjunto de objetos de una misma familia
- No es fácil soportar nuevos tipos de productos
  - Si se tiene que extender la interfaz de la Factoría abstracta

Factoría Abstracta

- Esquema, participantes y colaboraciones



Factoría Abstracta

- Implementación
- Factorías como *singletons*
  - En muchas ocasiones basta con una factoría concreta por cada familia de productos
- Creación de productos:
  - Método factoría por cada producto:
- La factoría abstracta sólo define una interfaz para crear productos, y es la subclase factoría concreta la que los crea. Ésta especificará sus productos redefiniendo el método factoría para cada uno.
- Prototipo para cada factoría concreta Se crea la factoría concreta inicializada con un objeto prototipo de cada producto. Los nuevos se crean haciendo clones del prototipo

- Definición de factorías extensibles

¿Qué ocurre cuando se quiere añadir un nuevo tipo de producto?

Una solución es añadir un parámetro a las operaciones que crean objetos para especificar el tipo de objeto que se va a crear. (Sólo haría falta una operación de creación que tendría este parámetro)

- Esta solución requiere también que el cliente haga narrowing o dynamic-cast

Aplicación en Swing

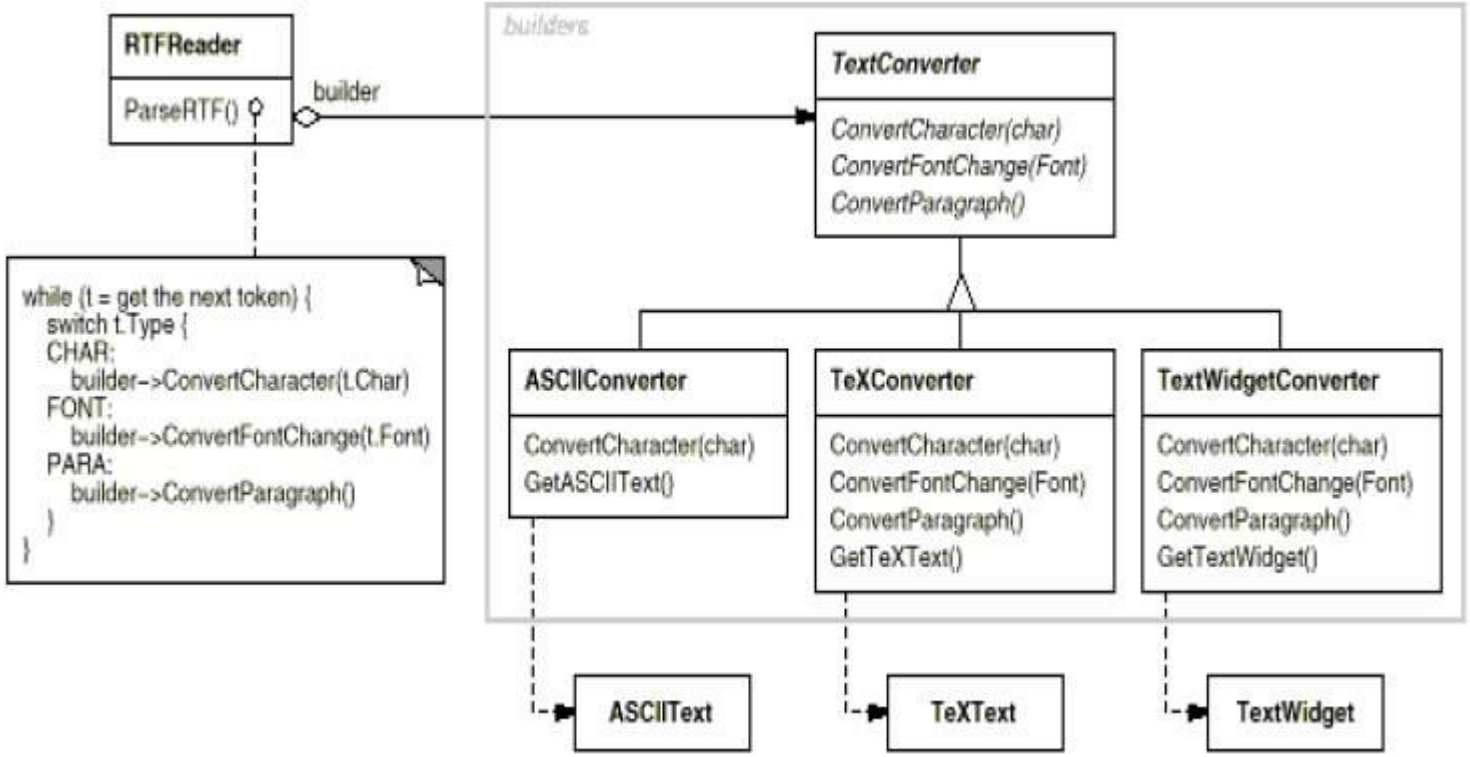
- Abstract factory : LookAndFeel
- Elección de la fábrica concreta : UIManager
- Producto abstracto : ComponentUI y sus clases derivadas (UIDelegate)
- Producto concreto : BasicListUI, MetalListUI…
- Selección de apariencia en Swing

```
private void seleccionarApariencia() {
    // Fuerza al sistema a adoptar una apariencia concreta
    String laf = UIManager.getSystemLookAndFeelClassName();
    try {
      UIManager.setLookAndFeel(laf);
    }
    catch (UnsupportedLookAndFeelException exc)
      {System.err.println("Unsupported: " + laf);}
    catch (Exception exc)
      {System.err.println("Error loading " + laf);}
}
```

Builder

- Propósito:
  - Permite a un cliente construir un objeto complejo especificando sólo su tipo y contenido, ocultándole todos los detalles de la construcción del objeto.
- Aplicación
  - Cuando el algoritmo para crear un objeto complejo debe ser independiente de las partes que constituyen el objeto y cómo se juntan
  - Cuando el proceso de construcción debe permitir representaciones diferentes para el objeto que se está construyendo

Builder


RTFReader
ParseRTF()
builder
builders
TextConverter
ConvertCharacter(char)
ConvertFontChange(Font)
ConvertParagraph()
while (t = get the next token) {
switch t.Type {
CHAR:
builder->ConvertCharacter(t.Char)
FONT:
builder->ConvertFontChange(t.Font)
PARA:
builder->ConvertParagraph()
}
}
ASCIIConverter
ConvertCharacter(char)
GetASCIIText()
TeXConverter
ConvertCharacter(char)
ConvertFontChange(Font)
ConvertParagraph()
GetTeXText()
TextWidgetConverter
ConvertCharacter(char)
ConvertFontChange(Font)
ConvertParagraph()
GetTextWidget()
ASCIIText
TeXText
TextWidget

Singleton

Propósito:

- Asegurar que una clase sólo tiene un ejemplar, y proporcionar un punto de acceso global a éste
- Motivación
  - Algunas clases sólo necesitan exactamente un ejemplar
  - Un *spooler de impresión en un sistema, aunque haya varias* impresoras
  - Un sólo sistema de archivos
  - Un sólo gestor de ventanas
  - • …
- En vez de tener una variable global para acceder a ese ejemplar único, la clase se encarga de proporcionar un método de acceso

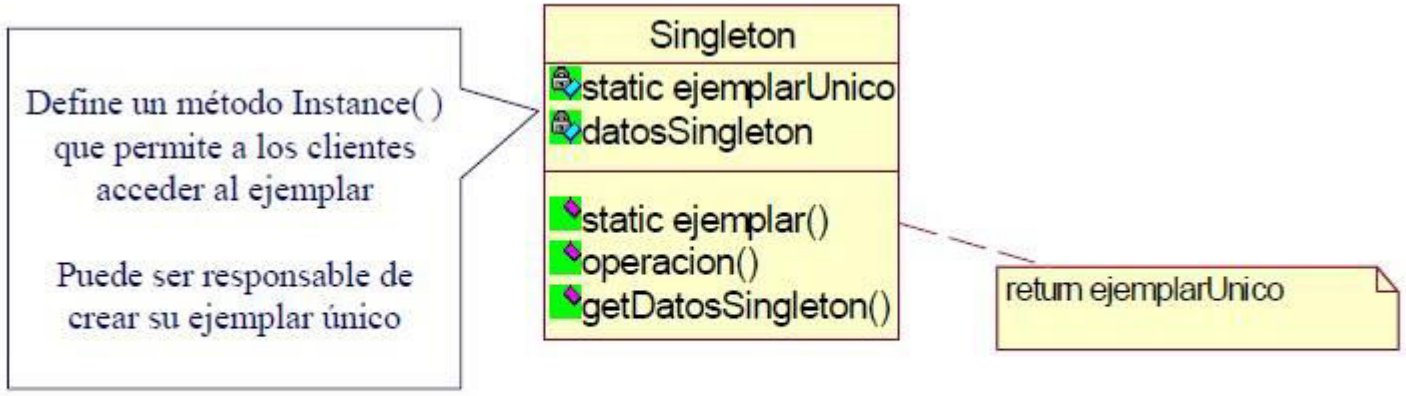
Singleton

Aplicación

- Cuando sólo puede haber un ejemplar de una clase, y debe ser accesible a los clientes desde un punto de acceso bien conocido
- Cuando el único ejemplar pudiera ser extensible por herencia, y los clientes deberían usar el ejemplar de una subclase sin modificar su código

Singleton

- Esquema, participantes y colaboraciones
  - Los clientes acceden al ejemplar de Singleton únicamente a través del método Instance de la clase Singleton



Patrones estructurales

- Lo fundamental son las relaciones de uso entre objetos
- Se trata de conseguir que cambios en los requisitos de la aplicación no ocasionen cambios en las relaciones entre los objetos
- Las relaciones de uso están determinadas por las ***interfaces que soportan los objetos***

Adaptador

Propósito:

- Convertir la interfaz de una clase en otra que esperan los clientes
- Otras denominaciones:
  - ClassAdapter y ObjectAdapter
  - Wrapper (Envolvente)

Motivación

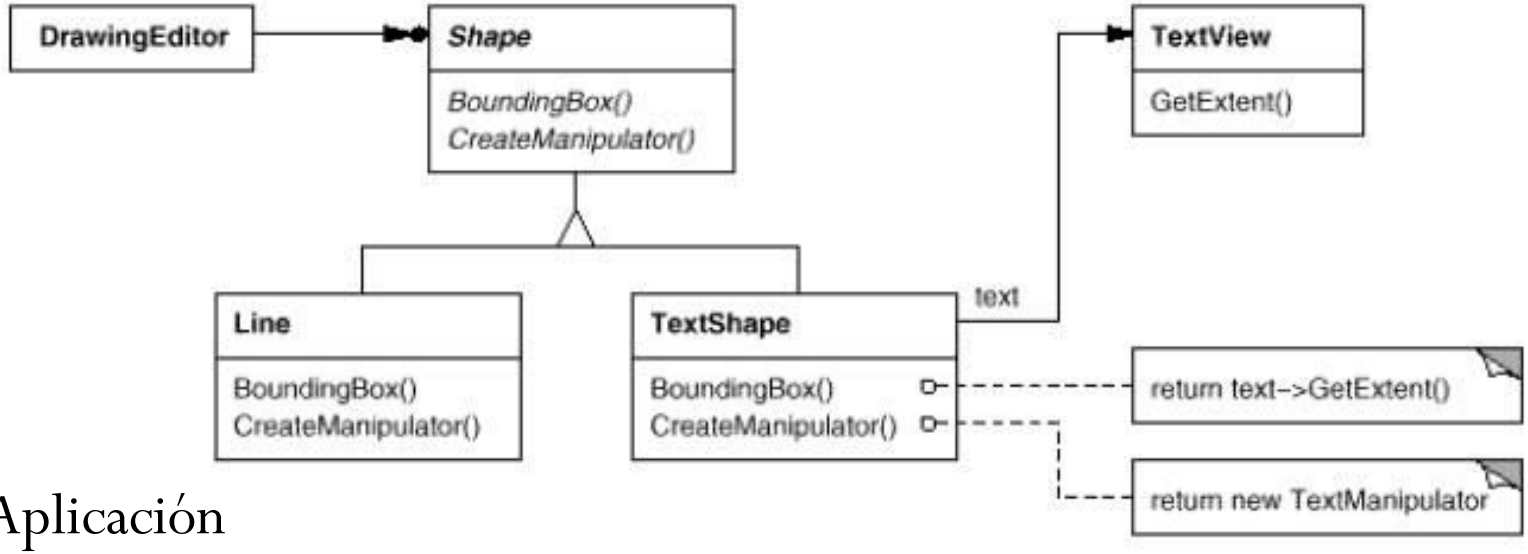
- Para reutilizar una clase de una biblioteca aunque su interfaz no correspondiera exactamente con el que requiere una aplicación concreta
  - En el ejemplo, la clase TextView
- Para añadir funcionalidad que la clase reutilizada no proporciona
  - En el ejemplo, la operación CreateManipulator()

Adaptador

- La clase TextShape adapta TextView a la interfaz Shape, de manera que el editor gráfico DrawingEditor puede reutilizar la clase TextView, que de otra manera sería incompatible.
- TextShape se puede hacer de dos maneras:

1) Heredando la interfaz Shape y la clase TextView (Adaptador de clases)
2) Componiendo un objeto TextView en un TextShape, que se implementa utilizando el objeto TextView (Adaptador de objeto)

Adaptador



Aplicación

Para usar una clase existente cuya interfaz no se corresponde con el que se necesita

Para crear una clase reutilizable que coopera con clases imprevistas (esto es, que no tienen necesariamente interfaces compatibles)

El adaptador de objeto sólo: para utilizar varias subclases existentes para las que sería poco práctico adaptar su interfaz heredando de cada una. Un adaptador de objeto puede adaptar la interfaz de su clase padre

Bridge

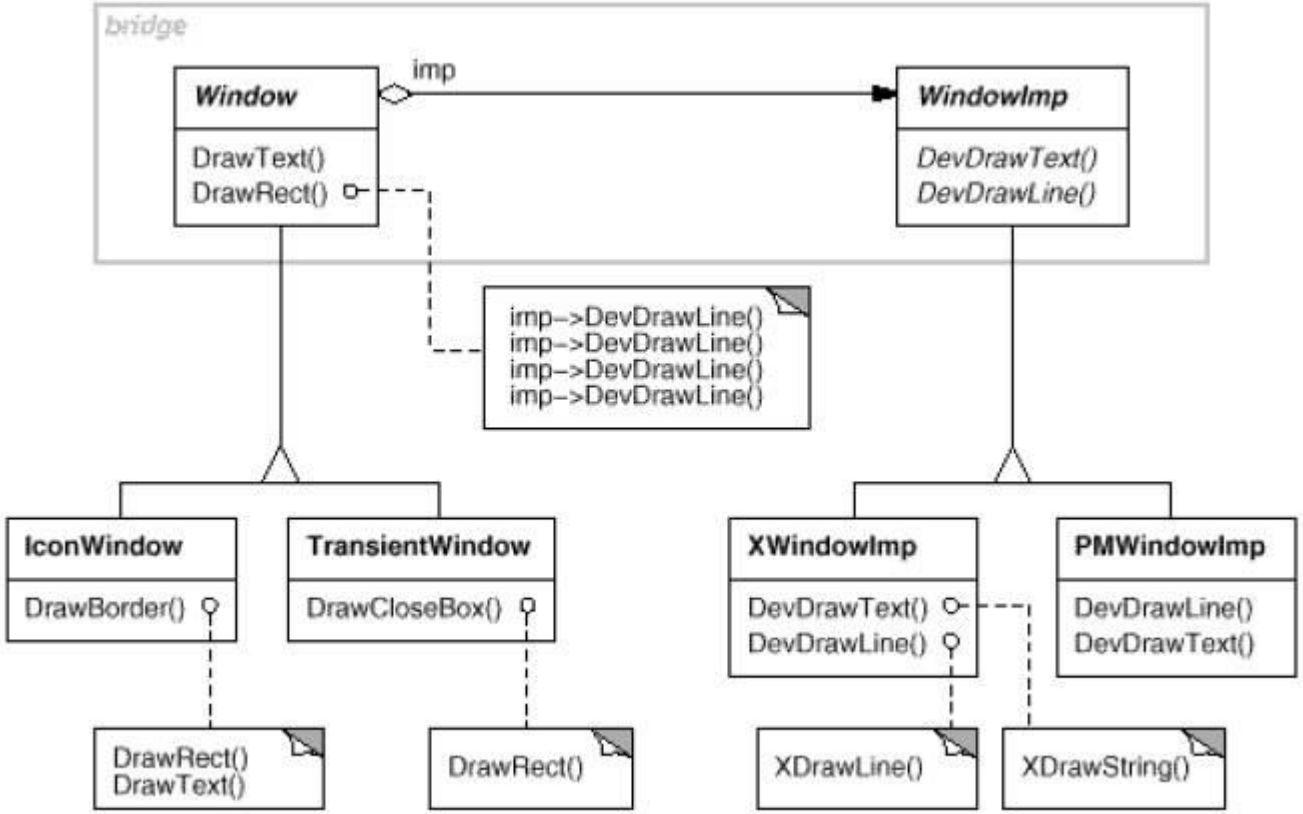
Propósito:

- Desacopla una abstracción de su implementación de manera que las dos puedan evolucionar independientemente
- Otras denominaciones:
  - Handle/Body

Motivación

- La herencia permite que una abstracción tenga varias implementaciones: esta relación se define en tiempo de compilación
- Hay casos en los que puede haber una gran jerarquía de clases de una abstracción (p.ej. Window, IconWindow, TransientWindow) y varias jerarquías de implementación (en diferentes plataformas)
  - *Es necesario desacoplar la abstracción de la implementación*

Bridge


bridge
Window
DrawText()
DrawRect()
imp
WindowImp
DevDrawText()
DevDrawLine()
imp->DevDrawLine()
imp->DevDrawLine()
imp->DevDrawLine()
imp->DevDrawLine()
IconWindow
DrawBorder()
TransientWindow
DrawCloseBox()
XWindowImp
DevDrawText()
DevDrawLine()
PMWindowImp
DevDrawLine()
DevDrawText()
DrawRect()
DrawText()
DrawRect()
XDrawLine()
XDrawString()

Bridge

Aplicación

- Para evitar una vinculación permanente entre una abstracción y su implementación
  - Permite cambiar la implementación en tiempo de ejecución
- Cuando tanto las abstracciones como las implementaciones podrían ser susceptibles de extensión mediante subclases
  - El patrón Bridge permite combinar las abstracciones e implementaciones y extenderlas independientemente
  - Permite además reducir la proliferación de clases
- Para que cambios en la implementación de una abstracción no hagan recompilar el código de los clientes
  - En C++ es más crítico porque la representación de una clase es visible en su definición
- Para compartir una implementación entre múltiples objetos, sin que lo noten los clientes

Proxy

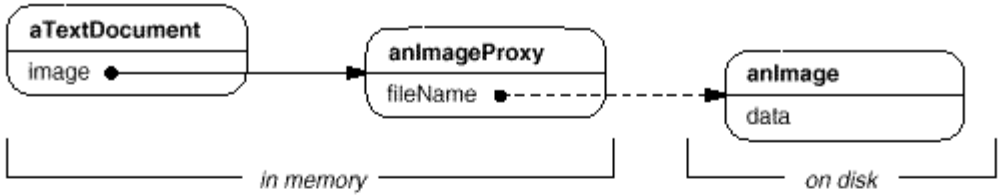
Propósito:

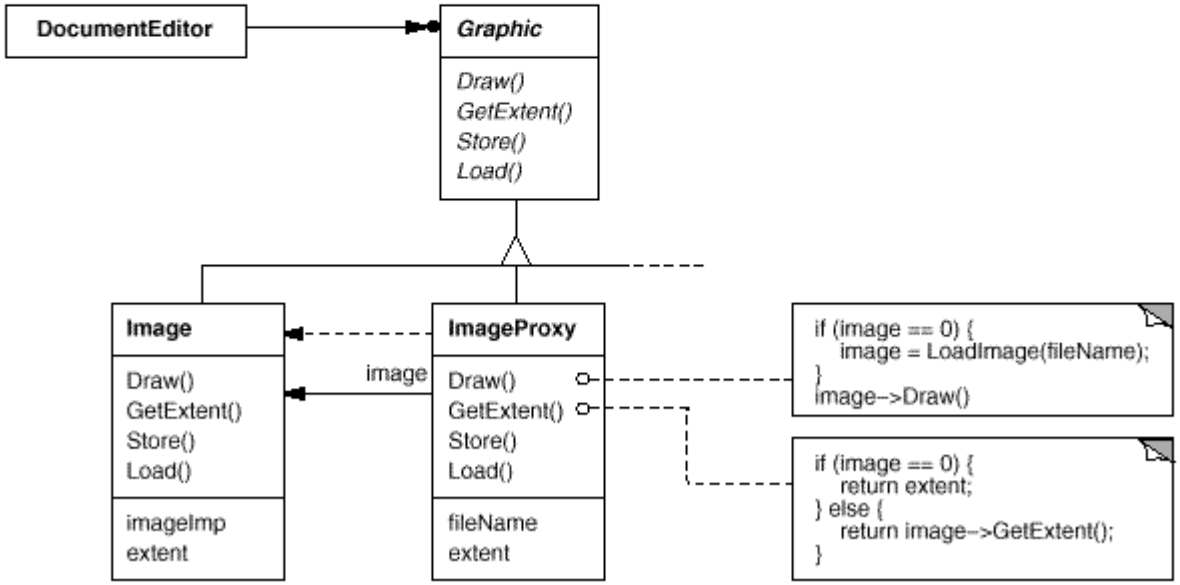
- Es un sustituto de otro objeto, para controlar el acceso a él
- Otras denominaciones:
  - Surrogate
  - Virtual proxy

Motivación

- Puede haber ocasiones en que se desee posponer el coste de la creación de un objeto hasta que sea necesario usarlo
- Al abrir un documento, postergar el crear una imagen hasta que se tenga que visualizar
- En programas de comunicaciones, para un objeto remoto
- El objeto proxy actúa en lugar del verdadero objeto, y ofrece las misma interfaz, y las solicita en el objeto cuando es necesario
- Por ejemplo cuando el editor del documento invoca la operación Draw en la imagen para visualizarla

Proxy


aTextDocument
image
anImageProxy
fileName
anImage
data
in memory
on disk


DocumentEditor
Graphic
Draw()
GetExtent()
Store()
Load()
Image
Draw()
GetExtent()
Store()
Load()
imageImp
extent
image
ImageProxy
Draw()
GetExtent()
Store()
Load()
fileName
extent
if (image == 0) {
image = LoadImage(fileName);
}
image->Draw()
if (image == 0) {
return extent;
} else {
return image->GetExtent();
}

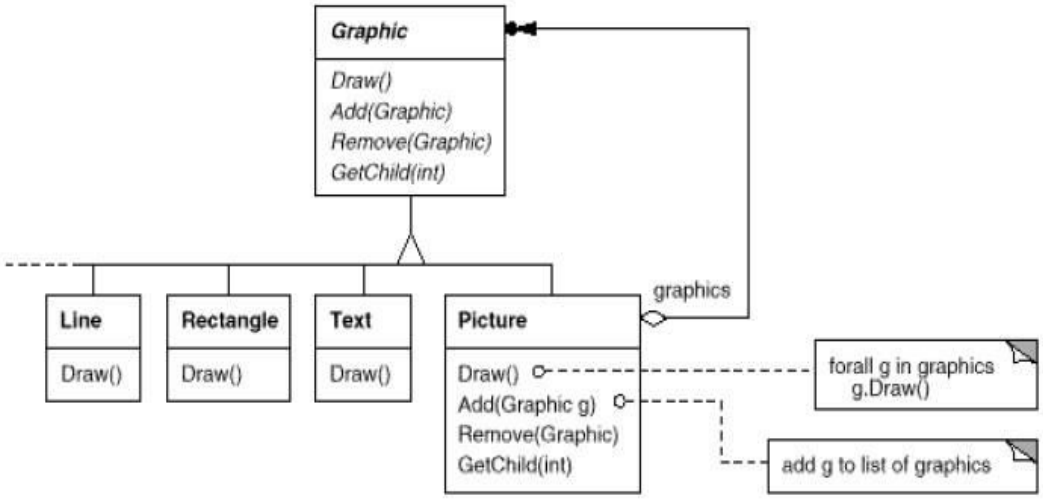
Composite

Propósito:

- Construir objetos complejos mediante la composición recursiva de objetos similares de manera similar a un árbol

Motivación

- Las aplicaciones gráficas tienen componentes que pueden agruparse para formar componentes mayores (contenedores)



Composite

Aplicación

- Para representar jerarquías de objetos parte-todo
- Para que los clientes puedan manejar indistintamente objetos individuales o composiciones de objetos
- Los clientes tratan todos los objetos en la estructura composite de manera uniforme

Decorador

Propósito:

- Asignar nuevas responsabilidades a un objeto dinámicamente. Es una alternativa a la creación de subclases por herencia
- Otras denominaciones:
  - Decorator
  - Wrapper

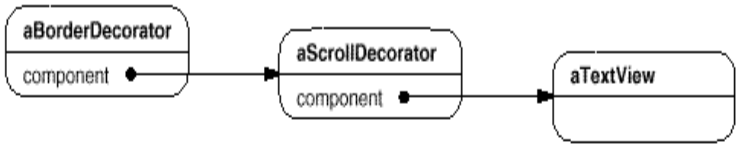
Motivación

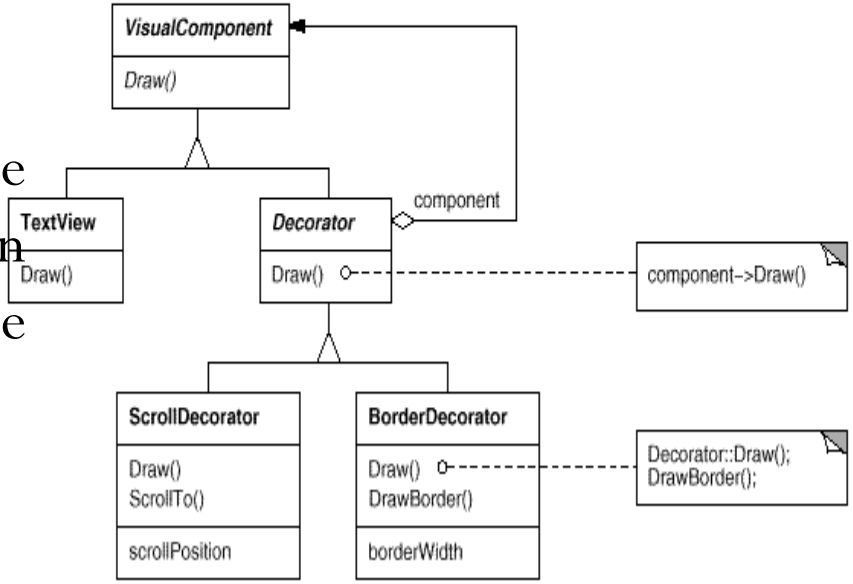
- Para añadir nuevas responsabilidades a objetos individuales y no a toda la clase
- Por ejemplo, una ventana de una interfaz gráfica puede tener bordes, scrolling, u otros artifactos
- Esto se puede hacer dinámicamente

Decorador

Aplicación

- Para asignar responsabilidades a objetos individuales de forma dinámica y transparente, sin afectar a otros objetos
- Para responsabilidades que pueden ser suprimidas
- Cuando no es práctico utilizar herencia de clases (porque se produciría una explosión del número de clases, o porque no se tiene la definición de la clase)





Fachada

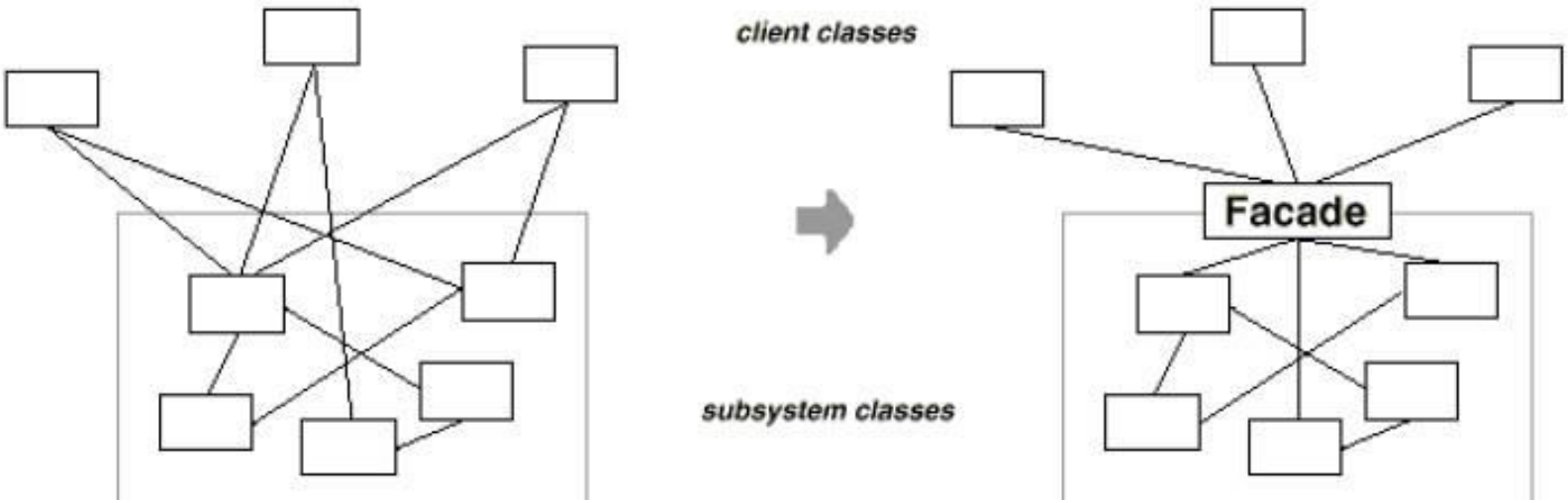
Propósito:

- Simplifica el acceso a un conjunto de objetos proporcionando uno que todos los clientes pueden usar para comunicarse con el conjunto
- Otras denominaciones:
  - Façade

Motivación

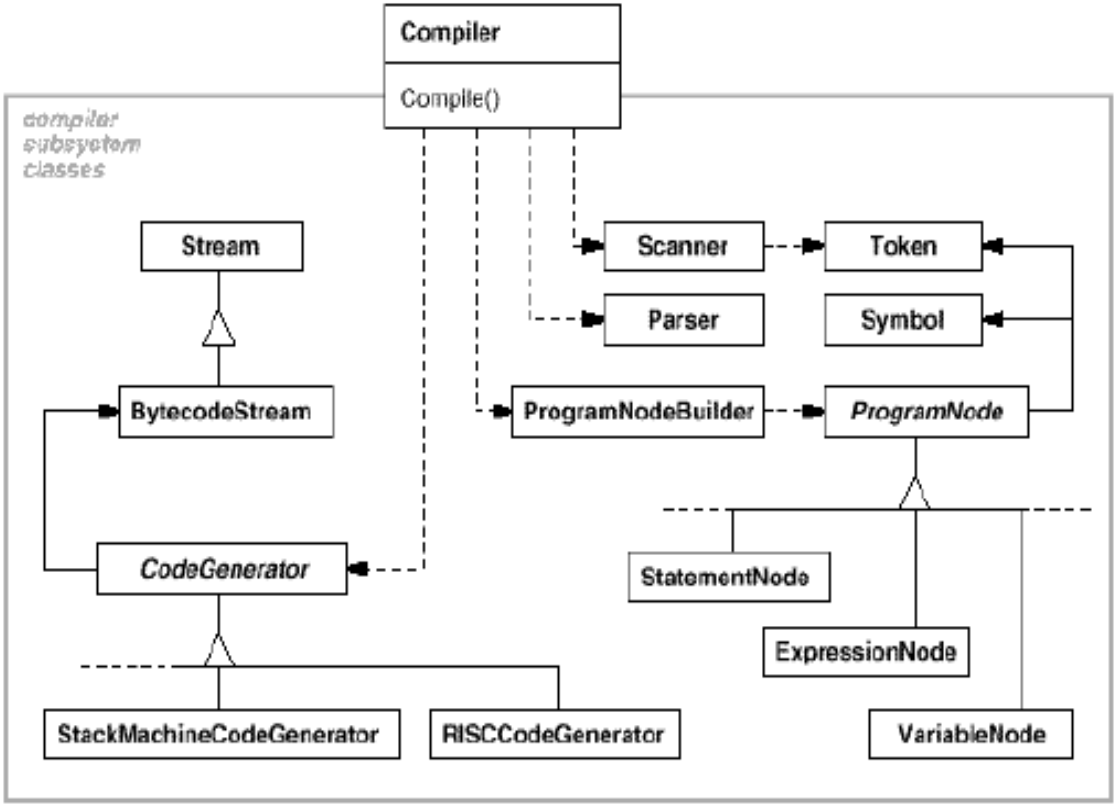
- Minimizar las comunicaciones y dependencias entre subsistemas

Fachada



Fachada

El subsistema de compilación ofrece scanner, parser, ... a través de una interfaz unificada (la fachada) de la funcionalidad del compilador, sin ocultar completamente las clases que lo implementan



Fachada

Aplicación

- Para proporcionar una interfaz sencilla a un subsistema complejo
- A medida que un subsistema evoluciona va teniendo más clases, más pequeñas, más flexibles y configurables
  - Hay clientes que no necesitan tanta flexibilidad y que quieren una visión más simple del subsistema
  - Sólo los clientes que necesiten detalles de más bajo nivel accederán a las clases detrás de la fachada
- Cuando hay muchas dependencias entre los clientes y las clases de implementación de una abstracción
  - La fachada desacopla el subsistema de los clientes y de otros subsistemas
  - Esto mejora la independencia de subsistemas y la portabilidad
- Para estructurar un sistema en capas
  - La fachada define el punto de entrada de cada nivel
  - Se pueden simplificar las dependencias obligando a los subsistemas a comunicarse únicamente a través de sus fachadas

Discusión

- Muchos patrones estructurales tienen bastante similitud en el esquema, los participantes y las colaboraciones
- Las diferencias están principalmente en el propósito

Adaptador y Bridge

- Ambos envían peticiones a un objeto cuya interfaz es distinta
- El adaptador trata de resolver incompatibilidades entre interfaces existentes, bridge trata de proporcionar una interfaz estable a los clientes, independiente de cómo evolucione la implementación
- Por tanto el Adaptador suele aparecer cuando ya hay clases desarrolladas, pero el Bridge es algo que el diseñador avanza al principio del ciclo de vida del sistema

Discusión

- Las diferencias están principalmente en el propósito

Composite y Decorator

- Ambos se basan en la composición recursiva para organizar un conjunto de objetos ilimitado
- El Decorador pretende facilitar el añadir responsabilidades a los objetos sin hacer subclases. Composite trata de que múltiples objetos se puedan tratar de manera uniforme, como uno (el énfasis está en la representación y no en el embellecimiento)
- Los propósitos son complementarios y por tanto se suelen usarlos dos patrones conjuntamente

Proxy

- Similar estructura que Decorator
- Especialización de Bridge

Patrones de comportamiento

- Algoritmos y asignación de responsabilidades entre objetos y patrones de comunicación entre objetos
- Distribución del comportamiento entre varias clases
- Mediante herencia
  - MétodoTemplate: definición abstracta de un algoritmo
  - Intérprete: representa una gramática como una jerarquía de clases
- Mediante composición de clases
  - Mediador: entre varios objetos para desacoplarlos
  - Cadena de responsabilidad
  - Observador (MVC)
- Encapsulado de comportamientos
  - Estrategia: encapsula un algoritmo en un objeto
  - Comando: encapsula una petición a un objeto
  - Estado: encapsula el estado de un objeto
  - Visitor: encapsula un comportamiento
  - Iterador: el método de recorrer una colección de clases

Patrones de comportamiento

- Chain of responsability
  - Petición delegada al proveedor de servicio responsable
- Command
  - Encapsula una petición como un objeto
- Interpreter
  - Intérprete de lenguaje para una gramática sencilla
- Iterator
  - Para acceder secuencialmente a elementos de una agregación
- Mediator
  - El mediador coordina las interacciones entre sus asociados
- Memento
  - Captura el estado de un objeto y lo restaura posteriormente

Patrones de comportamiento

- Observer
  - Para notificar automáticamente cambios de estado de un objeto a todos los dependientes
- State
  - Objeto cuyo comportamiento depende de un estado (el objeto puede aparentar cambiar de clase según un estado)
- Strategy
  - Abstracción para seleccionar uno o varios algoritmos
- Template method
  - Algoritmo con varios pasos suministrados por una clase derivada
- Visitor
  - Operaciones aplicadas a elementos de una estructura de objetos heterogénea

Método Template

Propósito:

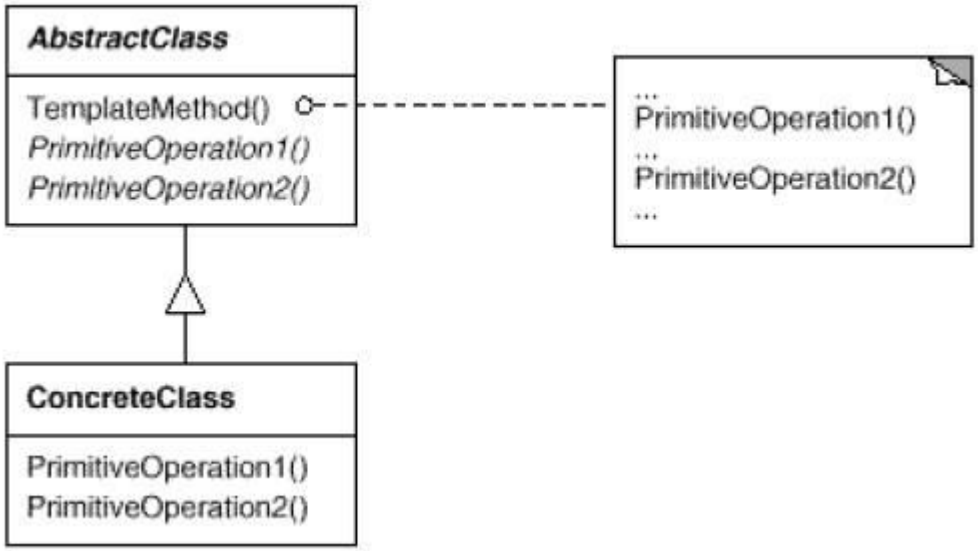
- Permite construir el esqueleto de un algoritmo dejando los detalles a las subclases

Aplicación

- Permite escribir las partes fijas de un algoritmo una sola vez y definir las partes que varían en las subclases
- Refactorización de código
  - Identificar las diferencias en el código existente
  - Separar las diferencias en nuevas operaciones
  - Reemplazar el código diferente con un método template que llama a esas operaciones nuevas
- Define los puntos de extensión de las subclases
  - Útil en frameworks: indica en qué puntos se puede extender

Método Template

- Esquema, participantes y colaboraciones
- La clase abstracta define las operaciones primitivas (pasos del algoritmo) que deben implementar las subclases concretas
- El método template define el esqueleto del algoritmo



Estrategia

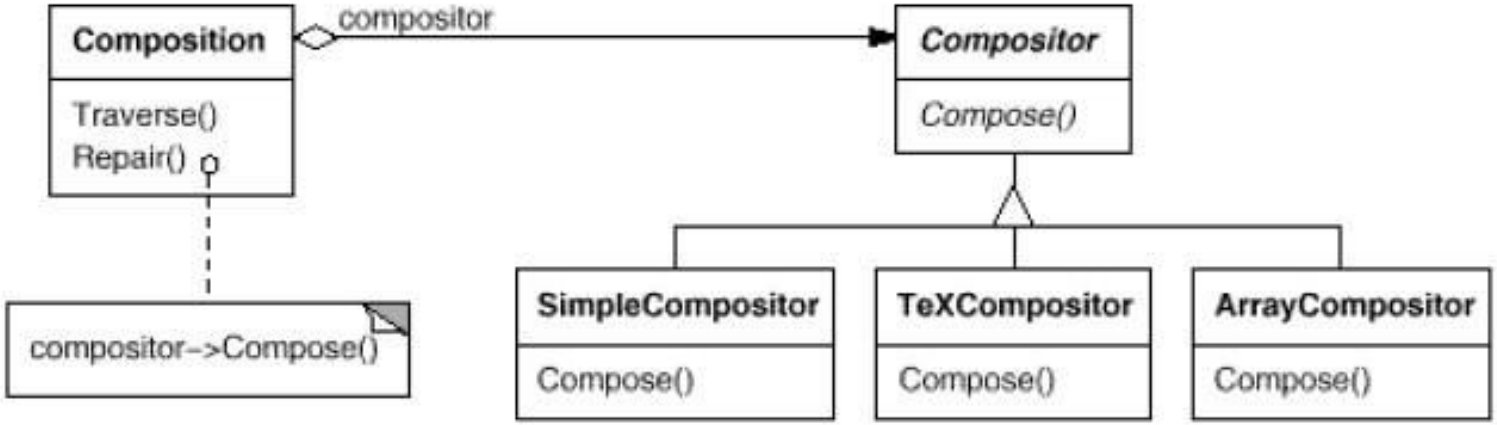
Propósito:

- Permite definir una familia de algoritmos, encapsulándolos de manera que se pueda intercambiar uno por otro
- Otras denominaciones:
  - *Policy (políticas)*

Motivación

- En un procesador de textos, una clase Composition se encarga de gestionar la división de líneas de un texto y para ello delega en
- Compositor el algoritmo de división (por líneas, por párrafos, o por un número fijo de caractere

Estrategia


Composition
Traverse()
Repair()
compositor
Compositor
Compose()
compositor->Compose()
SimpleCompositor
Compose()
TeXCompositor
Compose()
ArrayCompositor
Compose()

Estrategia

Aplicación

- Cuando hay muchas clases similares que sólo difieren en algún comportamiento
  - Se puede configurar una clase con varios comportamientos
- Se necesitan variantes de un algoritmo
- Para ocultar estructuras de datos complejas que usa el algoritmo
- En vez de utilizar muchas instrucciones condicionales poner los comportamientos alternativos en clases estrategia concretas

Estrategia

Esquema, participantes y colaboraciones

- La estrategia y el contexto colaboran para implementar un algoritmo concreto:
- La clase Strategy declara la interfaz común a todos los algoritmos
- El contexto se configura con una estrategia concreta y mantiene la referencia correspondiente
- El contexto puede definir una interfaz para que la clase estrategia acceda a sus datos (y le pasará a la estrategia su referencia) o bien le pasa todos los datos necesarios en las operaciones

Discusión

Template Method vs. Strategy

- El patrón método template utiliza herencia para variar parte de un algoritmo
- El patrón estrategia utiliza delegación para variar todo el Algoritmo
- Los métodos factoría se llaman frecuentemente por los métodos template

El futuro de los patrones.

- En la actualidad, se ha desarrollado y catalogado un número relativamente pequeño de patrones. De cualquier manera, los últimos cinco años han sido testigos de una tremenda explosión, en términos de interés industrial. Los patrones, junto con los componentes, ofrecen la esperanza de que la ingeniería de software, eventualmente, se parezca a otras disciplinas de ingeniería, con clases volviéndose el análogo en software de componentes electrónicos, y los patrones volviéndose el análogo en software de pequeños circuitos hechos de componentes.Antes de que esto suceda, existe aún una ingente cantidad de trabajo que llevar a cabo al identificar patrones y catalogarlos; también, se producirá esta situación, cuando el número de patrones existentes se vuelva tan grande, que se necesite una forma de indexado automática o semiautomática.

Muchas Gracias

- Consultas.
- La próxima clase…..

Bibliografía

- UML y Patrones, Craig Larman; Segunda Edición. Editorial: Prentice Hall.
- 'Ingeniería de software un enfoque práctico', Roger S. Pressman, 6th ed. Editorial MC Graw Hill. 2005
- 'Ingeniería del Software', Ian Sommerville. 7th ed. Editorial Prentice Hall.
- Apuntes Cátedra Análisis de Sistemas II, Facena UNNE. http://exa.unne.edu.ar/informatica/anasistem2/public_html/home.htm
- http://www.fdi.ucm.es/profesor/jpavon/poo/2.14PDOO.pdf